



Pashov Audit Group

Yuzu Money - July Security Review

July 7th 2026 - July 12th 2026



Executive Summary

A time-boxed security review of Yuzu Money - July was conducted through the Pashov Audit Group bug-hunt program. This report documents 8 issues across the reviewed codebase, including findings reported by 0x37, Kirkeelee, Tejas.

Project	Yuzu Money - July
Timeline	July 7th 2026 - July 12th 2026

About

Pashov Audit Group is a security research collective focused on smart-contract and protocol security. This report documents the issues surfaced during a time-boxed review delivered through our competitive bug-hunt program.

METHODOLOGY

Researchers independently reviewed the in-scope code and reported the issues they found. Each submission was then triaged and validated by the judging team; duplicates were merged and severities were finalized. Every accepted issue appears below with its impact and a recommended remediation. The client's response to each issue was then reviewed, and its resolution status - fixed, acknowledged, or invalidated - is recorded with the finding.

DISCLAIMER

A security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where the researchers try to find as many issues as possible. We can not guarantee 100% security after the review. Subsequent reviews, bug bounty programs and on-chain monitoring are strongly recommended. The client's remediation of each issue was reviewed after the hunt; the outcome is recorded in the finding's Resolution section.

Risk Classification

Critical

Loss or compromise of critical assets, or protocol-wide impact. Must be fixed before launch.

High

Significant material loss of assets, or serious harm to a group of users.

Medium

Moderate material loss, or moderate harm, often under conditional assumptions.

Low

Minor impact, limited scope, or requires unlikely conditions to trigger.

Info

No direct security impact - best-practice, code-quality or hardening suggestion.

Scope

The scope is all Solidity changes under `src/` here:

<https://github.com/Telos-Consilium/ouroboros-contracts/pull/27>

PR #27, head is at 3bf353e



For an issue to be valid, its root cause must be related to these changes.

Findings

SEVERITY	COUNT
Medium	3
Low	5
Total	8

ID	TITLE	SEVERITY	STATUS
M-01	A yzILP distribution that vests while total supply is zero is orphaned in `totalAssets`; the next depositor mints via the `supply == 0` branch and captures it at the order filler's expense	Medium	Fixed
M-02	PSMOVaultComposer redeem path is bricked by missing share approval to the PSM	Medium	Fixed
M-03	Same-block guard bypass enables atomic draining of immediate redemption liquidity	Medium	Fixed
L-01	A too-short distribution `period` vests within one block, letting a just-in-time depositor capture streaming yield meant for existing stakers (`minDistributionPeriod` defaults to `0`)	Low	Fixed
L-02	`PSMV2.maxRedeem` ignores the inner staked-vault redeem throttle that `redeem` enforces, so the view over-reports and `redeem` reverts at the boundary	Low	Fixed
L-03	Destination compose refunds use `tx.origin`, sending excess LayerZero fees to the executor	Low	Acknowledged
L-04	Filled redeem orders can become permanently stuck for blocklisted receivers and keep suppressing global liquidity	Low	Acknowledged
L-05	Visible NAV markdowns can be front-run by instant redemptions at the old NAV	Low	Acknowledged



Medium findings

[M-01] A yzILP distribution that vests while total supply is zero is orphaned in `totalAssets`; the next depositor mints via the `supply == 0` branch and captures it at the order filler's expense

MediumFixed

SUMMARY

yzILP prices shares off fee-adjusted total assets. Its `_convertToShares` (YuzulPV3.sol:320-335) has an ERC-4626-style fresh-vault branch:

```
uint256 supply = totalSupply();
if (supply == 0) {
    return assets * 10 ** _decimalsOffset(); // assets * 1e12
}
```

This branch assumes total assets are also ~0 whenever supply is 0. The protocol breaks that assumption because `totalAssets` includes reported-and-vesting distribution value that is independent state:

- `distribute(assets, period)` (YuzulPV3Facet.sol:218-235) does **not** pull any tokens. It records `lastDistributedAmount / Period / Timestamp`; the amount then vests linearly (`_distributedAssets`, YuzulPV3Facet.sol:509-518) into `totalAssets` via `_proxyGrossTotalAssets` -> `_proxyNetDistributedSinceUpdate` (YuzulPV3Facet.sol:432-435, 460-480). It is reported value, materialized as real collateral only when an order filler funds a redemption.
- The distribution accumulators are reset **only** by `updatePool` -> `_applyPoolUpdate` (YuzulPV3Facet.sol:344-349), never automatically when supply organically drains to zero via order fills.

So the following sequence orphans the distribution:

1. A distribution is scheduled and begins vesting.
2. Every share is redeemed through the order book (`createRedeemOrder` -> `fillRedeemOrder`), draining `totalSupply` to 0. If this happens before the distribution has vested, the exiting holders capture none of it and `redeemedDistributionsSinceUpdate` is not advanced against it.
3. Time passes; the distribution keeps vesting into `totalAssets` with **no shares in existence** to accrue it and no `updatePool` to clear it. It is now orphaned: `totalAssets > 0` while `totalSupply == 0`.

The first depositor after the drain calls `deposit`, whose `previewDeposit` -> `_convertToShares` takes the `supply == 0` branch and mints `netAssets * 1e12` shares — pricing solely off their own deposit and ignoring the orphaned `totalAssets`. `_applyPoolSizeCredit` (YuzulPV3Facet.sol:382-420) credits `poolSize` by only the deposit's worth (the `poolSize == 0` branch returns the yield-discounted deposit). The depositor now owns ~100% of a tiny supply while `totalAssets = poolSize(deposit) + orphan`. Redeeming those shares through the order book values them at `convertToAssets(shares) ~ totalAssets`, and `fillRedeemOrder` makes the filler front that full amount in real collateral. The depositor walks away with `deposit + orphan`; the filler (protocol) eats the orphan.



IMPACT

- **Theft of the full distribution.** The attacker's proceeds exceed their deposit by approximately the entire orphaned distribution amount, sourced from the order filler's fronted collateral. In the PoC, a `1e6` deposit returns `1001e6` — a net `1000e6` profit equal to the whole `1000e6` distribution.
- **Unprivileged extraction.** The capturing depositor needs no special role beyond the ordinary mint/redeem allowlist (`MINTER_ROLE` / `REDEEMER_ROLE` when restricted; none if not). They do not need `POOL_MANAGER_ROLE` or to have participated before the drain.
- **Attacker-influenceable reachability.** The only non-attacker input is that a distribution be active while supply hits zero. In a new or small pool the attacker can be the sole LP: deposit, wait for the operator's routine `distribute`, exit all shares (draining supply to 0), let the rest vest, re-enter cheap, and capture. `distribute` allows periods up to 7 days, giving a wide window.
- **No solvency guard catches it.** `totalAssets` legitimately reflects the vesting distribution; nothing reconciles it against `totalSupply == 0`. The value is simply mis-assigned to whoever mints first.

RECOMMENDATION

The root cause is that the `supply == 0` share-minting branch prices off the deposit alone while `totalAssets` can be non-zero. Close the seam so a fresh depositor cannot mint against orphaned value:

- **Preferred:** when `supply == 0` but `totalAssets() > 0` (orphaned distribution and/or residual pool), do not use the `assets * 1e12` shortcut. Either revert (forcing an operator `updatePool` to reconcile the orphaned value before deposits reopen) or price the mint against the existing `totalAssets` so the depositor cannot buy it for free.
- **And/or** reset the distribution accumulators when supply reaches zero, so orphaned value cannot persist across a drain — e.g. fold any un-accrued distribution into `poolSize` /cumulative counters at the fill that drives `totalSupply` to 0, or block the `supply == 0` fast-path while a distribution is in progress.
- Add a guard in `updatePool` /deposit that treats `totalSupply == 0 && totalAssets != 0` as a state requiring reconciliation rather than a fresh vault.

[M-02] PSMVaultComposer redeem path is bricked by missing share approval to the PSM

MediumFixed

LOCATION

PSMVaultComposer:_initializeShareToken:52-65

ROOT CAUSE

The composer is supposed to support both deposit and redeem flows against a PSM-backed vault whose share token is decoupled from the vault contract. For that setup to work, the composer must approve every contract that legitimately needs to pull tokens from it during execution.

The actual initialization only approves `SHARE_OFT` for the share token:



```
shareERC20 = IOFT(SHARE_OFT).token();
address vault1 = IPSM(address(VAULT)).vault1();
if (shareERC20 != vault1) revert ShareTokenNotVault(shareERC20, vault1);
if (!IOFT(SHARE_OFT).approvalRequired()) revert ShareOFTNotAdapter(SHARE_OFT);
IERC20(shareERC20).forceApprove(SHARE_OFT, type(uint256).max);
```

The inherited redeem path still calls `VAULT.redeem(..., address(this), address(this))`. In a PSM/PSMV2 vault, that redeem implementation pulls `vault1()` shares from the owner via `transferFrom`, so the PSM itself also needs allowance from the composer. Because that approval is missing, redeem execution always reverts.

This makes the PSM composer redeem leg non-functional. Users can move shares into the composer, but the composer cannot complete the vault redemption into underlying assets. Direct calls revert atomically, and cross-chain compose redemptions fail over into refunds instead of successful asset delivery.

IMPACT

All PSM-backed `redeemAndSend` flows fail. Direct redemptions revert and cross-chain compose redemptions refund instead of delivering assets, leaving the redeem leg permanently unusable until the composer is replaced or upgraded.

EXPLOIT PATH

1. A user calls `redeemAndSend(100, sendParam, refundAddress)` on a composer deployed with `VAULT = PSM/PSMV2`.
2. The composer pulls 100 `vault1()` shares from the user and then calls `VAULT.redeem(100, address(this), address(this))`.
3. `PSMV2._redeem()` executes `vault1().transferFrom(composer, psm, 100)`, but `allowance[composer][psm] = 0` because initialization only approved `SHARE_OFT`.
4. The transfer reverts, so the redeem flow never delivers underlying assets.

ADDITIONAL CONTEXT

- The redeem feature is non-functional because the composer contract itself holds the missing approval, so users cannot repair the failure externally.
- The same missing approval makes the redeem leg entirely nonfunctional and causes deterministic failure at the PSM `transferFrom` step.
- An apparent redeem quote can still be guaranteed to fail at execution, because the quote path ignores the missing composer-to-vault share allowance.
- If refund execution is underfunded after this deterministic redeem failure, already-delivered shares can remain stranded in the composer until someone retries with enough fee or the logic is patched.
- A narrower allowance model is also viable: the composer can approve the PSM on demand before redeeming and reset afterward.
- Setting the approval lazily right before `redeem` is an alternative if perpetual allowance to the vault is undesirable.

RECOMMENDATION

Fix 1: Approve the PSM vault for the decoupled share token during initialization.



```
function _initializeShareToken() internal virtual override returns (address shareERC20) {
    shareERC20 = IOFT(SHARE_OFT).token();

    address vault1 = IPSM(address(VAULT)).vault1();
    if (shareERC20 != vault1) revert ShareTokenNotVault(shareERC20, vault1);
    if (!IOFT(SHARE_OFT).approvalRequired()) revert ShareOFTNotAdapter(SHARE_OFT);

    IERC20(shareERC20).forceApprove(SHARE_OFT, type(uint256).max);
    IERC20(shareERC20).forceApprove(address(VAULT), type(uint256).max);
}
```

Fix 2: Override the redeem path to move shares into the PSM explicitly before invoking vault redemption, so execution does not depend on a later `transferFrom` by the PSM.

```
function _redeem(bytes32, uint256 shareAmount) internal virtual override returns (uint256
    assetAmount) {
    IERC20(SHARE_ERC20).safeTransfer(address(VAULT), shareAmount);
    assetAmount = VAULT.redeem(shareAmount, address(this), address(VAULT));
}
```

[M-03] Same-block guard bypass enables atomic draining of immediate redemption liquidity

Medium

Fixed

LOCATION

YuzuSameBlockGuard:_recordMintBlock/_checkSameBlockRedeem:30-45

ROOT CAUSE

The intended behavior of the same-block guard is to make mint-backed instant redemptions non-atomic. In liquidity-backed flows, that invariant is important because it prevents an attacker from using temporary capital to mint a position and immediately cash it out against the protocol's liquid reserves.

The implementation does not actually preserve that invariant. It records the current block only for the mint receiver and later checks only the redeem owner. Fresh shares can therefore be minted to one address, transferred in the same block, and redeemed from a second address with no guard hit:

```
function _recordMintBlock(address receiver, uint256 amount) internal {
    if (amount == 0 || _isSameBlockGuardExempt(receiver)) return;
    _getYuzuSameBlockGuardStorage()._lastMintBlock[receiver] = block.number;
}

function _checkSameBlockRedeem(address _owner) internal view {
    if (_isSameBlockGuardExempt(_owner)) return;
    if (_getYuzuSameBlockGuardStorage()._lastMintBlock[_owner] == block.number) {
        revert SameBlockMintRedeem(_owner);
    }
}
```

That bypass becomes high impact in integrations such as PSMV2 where deposits build backing in the vault stack but redemptions consume separate on-contract liquidity. An attacker can atomically rotate liquid



reserves into backing assets using flash liquidity, leaving the protocol solvent but its instant-withdraw path unusable until a trusted party rebalances the buffer.

IMPACT

A user can drain the contract's immediately redeemable liquidity in a single atomic transaction, causing instant redeem/withdraw functionality to become unavailable until operators replenish the buffer.

EXPLOIT PATH

1. The attacker flash-borrows X assets and calls `PSMV2.deposit(X, A)`, minting fresh shares to address A in block N.
2. PSMV2 records `lastMintBlock[A] = N`, while the deposited assets are converted into vault backing rather than remaining as redeemable PSM liquidity.
3. A transfers the fresh shares to address B in block N, and B calls `PSMV2.redeem(shares, B, B)`; the same-block check on B passes.
4. PSM pays X underlying from its local liquidity pool to B, the flash loan is repaid, and honest users lose access to instant redemption until liquidity is replenished.

ADDITIONAL CONTEXT

- There is also a caller-vs-receiver framing: stamping the mint receiver instead of the actor later checked on redeem leaves the guard principal-scoped incorrectly unless transfers are tracked.
- The same flaw is described in the specific PSMV2 deposit/redeem flow, where `deposit()` stamps `receiver` and `redeem()` checks `_owner`.
- The same issue is framed as a principal mismatch between the recorded mint identity and the enforced redeem identity after a transfer or helper-contract hop.
- This version emphasizes that any deployment using the helper while leaving shares transferable remains bypassable, even if the helper itself is wired correctly at mint and redeem call sites.
- The vulnerable transfer path can be implemented through a dedicated share-transfer hook rather than the generic ERC20 hook; the bug remains the same because transfers still fail to carry the stamp.
- There is also a more precise remediation angle: track same-block-minted balances and account for them across transfer and burn operations instead of using only an address timestamp.

RECOMMENDATION

Fix 1: Propagate the same-block stamp on transfers so freshly minted shares remain blocked after moving to another address.

```
function _update(address from, address to, uint256 value) internal override {
    super._update(from, to, value);
    if (value > 0 && to != address(0)) {
        uint256 stamp = YuzuSameBlockGuardV3Storage.layout()._lastMintBlock[from];
        if (stamp == block.number) {
            YuzuSameBlockGuardV3Storage.layout()._lastMintBlock[to] = block.number;
        }
    }
}
```

Fix 2: Add a redeem-side defense that also blocks shares received in the current block, not only shares minted to the current owner in the current block.



```
mapping(address => uint256) internal _lastReceiveBlock;

function _update(address from, address to, uint256 value) internal override {
    super._update(from, to, value);
    if (value > 0 && to != address(0)) {
        _lastReceiveBlock[to] = block.number;
    }
}

function _checkSameBlockRedeem(address owner) internal view {
    if (_lastReceiveBlock[owner] == block.number) revert SameBlockMintRedeem(owner);
}
```



Low findings

[L-01] A too-short distribution `period` vests within one block, letting a just-in-time depositor capture streaming yield meant for existing stakers (`minDistributionPeriod` defaults to `0`)

Low

Fixed

SUMMARY

`StakedYuzuUSDV3` streams rewards by transferring assets into the vault and vesting them linearly over `period`. `totalAssets()` (`StakedYuzuUSD.sol:91-94`, line 93) subtracts the not-yet-vested portion `_undistributedAssets()` (`StakedYuzuUSD.sol:382-393`; V2 guard `StakedYuzuUSDV2.sol:255-261`), so at `distribute` time the share price does not move — the reward accrues to the price only as it vests. This is the intended anti-front-running property: an extractor must remain in the vault across the vesting window (bearing dilution and exposure) to capture the reward.

The window can be collapsed to a single block:

- `reinitialize` (`StakedYuzuUSDV3.sol:51-74`) sets `maxDistributionPpm = type(uint256).max` (line 73) but **never writes** `minDistributionPeriod` (declared line 42), leaving it at its storage default of `0`.
- `distribute` (`StakedYuzuUSDV3.sol:242-253`) only enforces `period ≥ minDistributionPeriod` (lines 243-245); with the default that check is vacuous.
- The V1 floor is merely `period ≥ 1` (`StakedYuzuUSD.sol:98-100`).

With `period = 1`, `_undistributedAssets()` returns the full reward `R` at `elapsed = 0` (the distribution block) and `0` at `elapsed ≥ 1` (the next block), so `R` lands on the share price as a near-discrete jump one block later. A depositor who enters in the **same block** as `distribute` — after the reward is funded but before it vests — mints shares at the pre-jump price and owns a $D / (T_0 + D)$ fraction of `R` one block on. `initiateRedeem` (`StakedYuzuUSDV3.sol:220-239`) then fixes `order.assets` at the post-jump price into `totalPendingOrderValue` via `_initiateRedeem` (`StakedYuzuUSD.sol:341-370`, line 349) and pays it out unchanged in `_finalizeRedeem` (`StakedYuzuUSD.sol:372-376`), so the extractor bears no post-vest exposure (and, when instant redeem is enabled, needs no wait at all).

The result: a reward meant for stakers present before the distribution is siphoned by a one-block JIT depositor, in proportion to the capital they flash in.

IMPACT

- **Yield theft from honest stakers.** The reward `R` is intended to accrue to holders who were staked before the distribution. A JIT depositor with principal `D` captures $\sim R \cdot D / (T_0 + D)$ of it for a single block of capital-at-risk; a whale with $D \gg T_0$ captures almost all of `R`. Pre-existing stakers are diluted to $\sim R \cdot T_0 / (T_0 + D)$.
- **Unprivileged, repeatable, low-risk.** The extractor needs no role — only the ability to land a transaction in the same block as (or immediately bundled after) the `distribute` tx, which is observable in the mempool. `initiateRedeem` /instant redeem locks in the vested price, so there is no downside once the reward has vested.
- **The streaming defense is defeated, not merely weakened.** This is not "some MEV is possible"; a `period = 1` distribution provides essentially the same front-runnability as an unstreamed direct donation. The mechanism that is supposed to prevent exactly this contributes nothing at a sub-block period.



- **No solvency or custody impact.** No assets are created or lost and the vault stays fully solvent (`balanceOf(this) ≥ totalPendingOrderValue + undistributed` holds throughout); the harm is a redistribution of intended yield among participants.
- **Gated on operator period choice.** At a realistic `period` (hours/days) the attack degrades to ordinary proportional staking — the JIT depositor must hold real capital across the window and bear the associated risk/dilution. The defect is the absence of a floor: nothing forces the period high enough for the streaming property to hold.

RECOMMENDATION

Enforce a meaningful minimum distribution period at the protocol level so safety does not depend on operator discipline or an out-of-band configuration call:

1. **Seed a non-zero `minDistributionPeriod` in `reinitialize`** (mirroring how it already seeds `maxDistributionPpm`), e.g.: `minDistributionPeriod = 1 days; // or the smallest window operations will ever use`
2. **And/or enforce a hard floor in `distribute`** independent of the configurable minimum, so no configuration (including a later `setMinDistributionPeriod(0)`) can reopen the sub-block path: `if (period < MIN_DISTRIBUTION_PERIOD_FLOOR) revert DistributionPeriodTooLow(period, MIN_DISTRIBUTION_PERIOD_FLOOR);` where `MIN_DISTRIBUTION_PERIOD_FLOOR` is comfortably larger than a block (e.g. `1 hours`).
3. **Add a lower bound to `setMinDistributionPeriod`** (`StakedYuzuUSDV3.sol:334-341`), which today only rejects `> 7 days` and otherwise accepts `0`, so the guard cannot be silently disabled after deployment.

Any one of these independently closes the path; (1)+(2) together give defense in depth and remove the dependency on the deployment runbook remembering to configure the guard. Keep `test/DistributeTerminateRescuePoC.t.sol` as a regression test — after the fix, `distribute` with `period = 1` should revert `DistributionPeriodTooLow`, and the short-period JIT test should be updated to assert the reverting behavior.

[L-02] `PSMV2.maxRedeem` ignores the inner staked-vault redeem throttle that `redeem` enforces, so the view over-reports and `redeem` reverts at the boundary

Low

Fixed

SUMMARY

`PSMV2` wraps two vaults (`vault0` -> `vault1`, the staked yzUSD vault) and redeems by pulling the owner's `vault1` shares into the PSM and calling `_vault1.redeem(shares, address(this), address(this))` (`PSMV2.sol:139`). That inner redeem runs `StakedYuzuUSDV3.redeem`, which enforces `maxRedeem(_owner = PSM)` and reverts `ERC4626ExceededMaxRedeem` when the request exceeds the staked vault's PSM-keyed **redeem throttle** (`StakedYuzuUSDV3.sol:210-213`, throttle folded at 149-155).

The PSM redeem view never accounts for that inner throttle. `PSM.maxRedeem` (`PSM.sol:151-161`) sizes capacity from only the owner's `vault1` balance and the PSM's liquidity buffer; `PSMV2.maxRedeem` (`PSMV2.sol:74-79`) adds only the PSM's own end-user-keyed redeem throttle. Neither calls `_vault1.maxRedeem(address(this))` nor otherwise reads the inner redeem-throttle remaining. When that inner throttle is the binding constraint, `PSM.maxRedeem(owner)` returns a value strictly larger than what `PSM.redeem` can actually settle, and a redeem for the reported maximum reverts.



This is asymmetric with the deposit side by construction: `PSM.maxDeposit` (PSM.sol:145-147) computes `min(_vault0.maxDeposit(this), _vault0.convertToAssets(_vault1.maxDeposit(this)))`, so it does fold in the inner vaults' mint throttles (`StakedYuzuUSDV3.maxDeposit` includes the mint throttle). The redeem view drops the equivalent inner limit.

IMPACT

- **ERC-4626 conformance break.** The standard requires `redeem(maxRedeem(owner), ..., owner)` not to revert. Under a bound inner redeem throttle it reverts with `ERC4626ExceededMaxRedeem(PSM, requested, innerCap)`.
- **Integration availability / DoS at the boundary.** Any contract that reads `PSM.maxRedeem` to size a redemption (routers, zap/unwind helpers, liquidation bots, vault aggregators) will build transactions that revert once the staked vault's global redeem throttle is near-exhausted, until the staked vault's daily window resets. Direct users can retry with a smaller amount; composed integrations often cannot and simply fail.
- **No solvency or fund-loss impact.** The throttle is still fully enforced — value never escapes the cap. This is a view/execution divergence (availability + spec compliance), not a theft or over-redemption bug.
- **Realistic trigger.** The PSMV2 NatSpec (PSMV2.sol:125-127) documents that the inner owner-keyed limits are intended to bind the PSM (the PSM redeems as `address(this)`), so the precondition — the PSM not holding `THROTTLE_EXEMPT_ROLE` in the staked vault — is the intended deployment, not a misconfiguration. The issue is moot only if operations exempt the PSM inside the inner vaults (in which case the inner throttle never binds).

RECOMMENDATION

Make `PSM.maxRedeem` fold in the inner (`vault1`) redeem-throttle capacity, symmetric with how `PSM.maxDeposit` already folds in the inner mint throttles.

Important subtlety: **do not** implement this as `Math.min(super.maxRedeem(owner), _vault1.maxRedeem(address(this)))`. Between operations the PSM holds no `vault1` shares, so `_vault1.maxRedeem(PSM)` reads `0` (bounded by the PSM's balance), which would wrongly clamp `maxRedeem` to `0`. The cap that actually binds only materializes mid-execution, when the pulled shares are transiently held by the PSM; its value is `_vault1.convertToShares(innerRedeemThrottleRemaining)`.

Concretely:

- Expose the staked vault's redeem-throttle remaining as a view (the counter is global / not per-owner, so a parameterless or address-agnostic `redeemThrottleRemaining()` on `StakedYuzuUSDV3` suffices; the struct is already retrievable via `getRedeemThrottle()`).
- In `PSM.maxRedeem` (or the `PSMV2` override), bound the returned shares additionally by `_vault1.convertToShares(innerRedeemThrottleRemaining)` before applying the PSM's own liquidity and throttle limits, so the view can never exceed what `_vault1.redeem(..., PSM, PSM)` will accept.

This restores the ERC-4626 `redeem(maxRedeem(owner), ..., owner)` parity invariant and makes the redeem view consistent with the deposit view. Keep `test/ThrottleAuditPoC_0.t.sol` as a regression test (flip the final expectation once the view is corrected so that `psm.redeem(psm.maxRedeem(user), ...)` succeeds).

[L-03] Destination compose refunds use `tx.origin`, sending excess LayerZero fees to the executor



Low

Acknowledged

LOCATION

PSM0VaultComposer:constructor:23-25

ROOT CAUSE

The compose flow should preserve user ownership of any unused destination-chain messaging budget. A user funds the initial omnichain operation, and any surplus native fee from the destination-side follow-up send should return to that user or to an explicitly provided refund address.

The current implementation instead uses the destination transaction's `tx.origin` as the refund recipient. In the inherited compose logic, both the success path and the catch path forward `tx.origin` into the OFT send/refund flow:

```
if (_oftIn == ASSET_OFT) {
  _depositAndSend(_composeFrom, _amount, sendParam, tx.origin, msg.value);
} else {
  _redeemAndSend(_composeFrom, _amount, sendParam, tx.origin, msg.value);
}
```

On a compose transaction, `tx.origin` is the executor or manual caller that delivered the LayerZero compose message, not the user encoded in the message payload. The bridged tokens still go to the intended recipient, but any excess native messaging fee is refunded to an unrelated external actor.

This creates a repeatable user-loss bug on every overfunded compose execution. It also affects the failure path: token refunds still route correctly through the message-derived destination, but any native fee surplus from that refund send is again paid to the executor because the composer uses the wrong refund identity.

IMPACT

Users lose any excess destination-chain messaging fee on successful second-hop sends, and the same misdirection applies to surplus native fee on compose-failure refund sends.

EXPLOIT PATH

1. A user initiates a cross-chain PSM action and funds the destination compose with more native value than the exact second-hop OFT send requires.
2. A LayerZero executor delivers the compose message on the destination chain, making the executor EOA equal to `tx.origin` inside the composer.
3. The composer calls `_depositAndSend` or `_redeemAndSend` and passes `tx.origin` as the OFT `refundAddress`.
4. The OFT/endpoint refunds unused native fee to the executor, and the user permanently loses that surplus.

RECOMMENDATION

Fix 1: Use the original compose sender instead of `tx.origin` when deriving the refund recipient.



```
function LzCompose(  
    address _composeSender,  
    bytes32 _guid,  
    bytes calldata _message,  
    address,  
    bytes calldata  
) public payable override {  
    address refundAddr = OFTComposeMsgCodec.composeFrom(_message).bytes32ToAddress();  
  
    try this.handleCompose{value: msg.value}(_composeSender,  
        OFTComposeMsgCodec.composeFrom(_message), OFTComposeMsgCodec.composeMsg(_message),  
        OFTComposeMsgCodec.amountLD(_message), refundAddr) {  
        emit Sent(_guid);  
    } catch (bytes memory _err) {  
        _refund(_composeSender, _message, OFTComposeMsgCodec.amountLD(_message), refundAddr,  
            msg.value);  
        emit Refunded(_guid);  
    }  
}
```

Fix 2: Add an explicit `refundAddress` field to the composed payload and use that field for both success and failure flows.

```
(address refundAddress, SendParam memory sendParam, uint256 minMsgValue) =  
abi.decode(_composeMsg, (address, SendParam, uint256));  
  
if (_oftIn == ASSET_OFT) {  
    _depositAndSend(_composeFrom, _amount, sendParam, refundAddress, msg.value);  
} else {  
    _redeemAndSend(_composeFrom, _amount, sendParam, refundAddress, msg.value);  
}
```

[L-04] Filled redeem orders can become permanently stuck for blocklisted receivers and keep suppressing global liquidity

Low

Acknowledged

LOCATION

YuzuProto:rescueTokens:244-253

ROOT CAUSE

The redeem-order flow is supposed to reserve assets at fill time and release them at finalization to the order's designated receiver, while preserving the remaining liquidity buffer for instant withdrawals. If finalization cannot succeed, the protocol should still provide some way to complete or recover that reserved payout without leaving the reservation permanently live.

Instead, a filled order is finalized through a push-only transfer to a fixed `receiver`, and the protocol offers no reroute or recovery path if that transfer becomes impossible. The underlying asset cannot be rescued at all, and the order cannot be canceled once it has reached `Filled`. The relevant changed logic hard-blocks rescue of the configured asset and always subtracts filled-order reservations from liquid capacity:



```
function rescueTokens(address token, address to, uint256 amount) external virtual onlyRole(-
ADMIN_ROLE) {
    if (token == address(this)) {
        uint256 outstandingBalance = balanceOf(address(this)) - totalPendingOrderSize();
        if (amount > outstandingBalance) revert ExceededOutstandingBalance(amount,
        outstandingBalance);
    } else if (token == _asset) {
        revert InvalidAssetRescue(token);
    }
    SafeERC20.safeTransfer(IERC20(token), to, amount);
}
```

This becomes exploitable whenever the underlying token has real-world transfer restrictions such as blocklisting. A filled order then becomes permanently stuck, and because its payout remains counted in `totalUnfinalizedOrderValue`, the protocol keeps understating available liquid assets for everyone else. The result is a persistent availability failure that can scale from one user's trapped redemption to protocol-wide instant-withdraw degradation.

IMPACT

A single filled order can become permanently unclaimable, and its reserved assets remain subtracted from the protocol's liquid buffer. Large stuck orders can materially reduce or even freeze instant withdrawals until governance deploys a manual recovery upgrade.

EXPLOIT PATH

1. A user calls `createRedeemOrder(...)` with receiver `R`, and an authorized filler later calls `fillRedeemOrder(orderId)` so the order becomes `Filled` and its payout is added to `totalUnfinalizedOrderValue`.
2. Before the owner/controller calls `finalizeRedeemOrder(orderId)`, the underlying asset blocks transfers to `R`.
3. `finalizeRedeemOrder(orderId)` reverts during `SafeERC20.safeTransfer(asset, R, order.assets)`, so the order stays `Filled` and the reservation remains in accounting.
4. Because the order can no longer be canceled and `rescueTokens(asset, ...)` always reverts, the reserved amount remains trapped and continues shrinking `liquidityBufferSize()` for all instant redeemers.

RECOMMENDATION

Fix 1: allow receiver replacement for filled orders before settlement.

This preserves the existing accounting model and only changes the destination when the original receiver can no longer accept funds.

```
function updateFilledOrderReceiver(uint256 orderId, address newReceiver) external {
    Order storage order = _getOrder(orderId);
    if (msg.sender != order.owner && msg.sender != order.controller && !hasRole(ADMIN_ROLE,
    msg.sender)) revert Unauthorized();
    if (order.status != OrderStatus.Filled) revert OrderNotFilled(orderId);
    if (newReceiver == address(0)) revert InvalidZeroAddress();
    order.receiver = newReceiver;
}
```



Fix 2: add an emergency finalizer that settles a filled order to a replacement address and clears the reservation.

This gives governance a narrowly-scoped recovery tool without opening general asset rescue for the underlying token.

```
function emergencyFinalizeRedeemOrder(uint256 orderId, address replacementReceiver)
external
onlyRole(ADMIN_ROLE)
{
    Order storage order = _getOrder(orderId);
    if (order.status != OrderStatus.Filled) revert OrderNotFilled(orderId);
    if (replacementReceiver == address(0)) revert InvalidZeroAddress();

    order.status = OrderStatus.Finalized;
    _getYuzuOrderBookStorage()._totalUnfinalizedOrderValue -= order.assets;
    SafeERC20.safeTransfer(IERC20(asset()), replacementReceiver, order.assets);
}
```

[L-05] Visible NAV markdowns can be front-run by instant redemptions at the old NAV

Low

Acknowledged

LOCATION

YuzuIssuer:redeem:157-167

ROOT CAUSE

Instant redemption should either settle against the latest intended valuation or be temporarily blocked while a known valuation decrease is being applied. That prevents one holder from exiting at a price that the protocol already knows is stale while other holders remain exposed to the markdown.

Instead, redemption uses the valuation that exists in storage at the instant the redeem transaction executes. There is no coupling between a pending NAV update and redemption settlement:

```
function redeem(uint256 tokens, address receiver, address owner) public virtual returns (uint256)
{
    uint256 maxTokens = maxRedeem(owner);
    if (tokens > maxTokens) {
        revert ExceededMaxRedeem(owner, tokens, maxTokens);
    }
    (uint256 assets, uint256 fee) = _previewRedeem(tokens);
    _withdraw(_msgSender(), receiver, owner, assets, tokens, fee);
}
```

A publicly visible markdown therefore creates a window where redemptions still use the old, higher NAV.

That window enables an informed holder to exit ahead of the markdown and receive more assets than the post-update valuation supports. The protocol does not lose track of balances, but value is redistributed from remaining holders to the early exiter purely through transaction ordering around a predictable price change.



IMPACT

A holder with mempool visibility can avoid an upcoming markdown and redeem at an inflated pre-markdown price, depleting liquidity at the expense of holders who remain after the NAV update.

EXPLOIT PATH

1. NAV is 1.00, instant redemption is enabled, and a holder owns 1,000 redeemable shares while the buffer has sufficient liquidity.
2. A NAV manager submits `setNav(0.90e18)` and the transaction becomes visible in the public mempool.
3. The holder front-runs with `redeem(1_000e12, holder, holder)` and redemption executes first, so `_previewRedeem` still values the shares at the old NAV.
4. The holder receives roughly 1,000 assets minus fees, then the markdown executes and remaining holders bear the reduced 0.90 NAV.

RECOMMENDATION

Fix 1: Block instant redemption while a NAV update is pending or being applied.

This is the least invasive mitigation if the protocol can tolerate a brief redemption pause around NAV changes.

```
bool public navUpdateInProgress;

function setNav(uint256 newNav) external onlyRole(NAV_MANAGER_ROLE) {
    navUpdateInProgress = true;
    _setNavInternal(newNav);
    navUpdateInProgress = false;
}

function redeem(uint256 shares, address receiver, address owner) public override returns (uint256)
{
    require(!navUpdateInProgress, "nav update in progress");
    return super.redeem(shares, receiver, owner);
}
```

Fix 2: Introduce a pending NAV state that disables instant redemption as soon as the lower NAV is announced, then activates it after the update window.

This removes the stale-price exit window without relying on private transaction submission.

```
uint256 public pendingNav;
uint256 public pendingNavActivateTime;

function announceNav(uint256 newNav, uint256 activateTime) external onlyRole(NAV_MANAGER_ROLE) {
    pendingNav = newNav;
    pendingNavActivateTime = activateTime;
}

function canRedeem(address owner) public view override returns (bool) {
    if (pendingNavActivateTime != 0 && block.timestamp < pendingNavActivateTime) return false;
    return super.canRedeem(owner);
}
```